# A Multi-Prover Architecture for Verifying Foundational Properties of AI Systems

With an Application to the Apeiron Framework

David Beniers

*Independent Researcher, Antwerp, Belgium*

david.beniers@telenet.be

April 24, 2026

**Abstract**

We present a multi-prover *architecture* for formally verifying the atomicity—the irreducibility—of elementary representational units within the Apeiron Framework. The architecture integrates four theorem provers (SymPy, Z3, Lean 4, and Coq) into a unified pipeline, `self_proving.py`, that generates proof obligations, executes them, and exports JSON certificates.

A rigorous evaluation on a benchmark of 47 observables reveals the current formal coverage: **Z3** correctly decides Boolean atomicity for integers and sets; **Lean 4** and **Coq** provide genuine proofs for integers via computational reflection, but for all other observable types and frameworks their backends are still placeholders; **SymPy** serves as a fast consistency filter but is not sound. We formalise this situation as a *trust hierarchy* and generate certificates that faithfully reflect it.

The contribution of this paper is the architecture itself, together with an honest characterisation of what each prover can currently deliver. We discuss the semantic gap induced by string-templating, propose an AST-based translation layer as the next step, and outline how the Lean/Coq backends can be extended to broader classes of observables.

## Contents

**1 Introduction** **3**

**2 Background: The Apeiron Framework and Multi-Axial Atomicity** **5**

2.1 The Apeiron Framework . . . . . 5

2.2 Multi-Axial Atomicity . . . . . 5

**3 The Multi-Prover Verification Framework** **7**

3.1 Overview of `self_proving.py` . . . . . 7

3.2 Translation Semantics and Prover Encodings . . . . . 7

3.2.1 Formal Specification of Multi-Axial Atomicity . . . . . 7

3.2.2 Encoding Across Provers . . . . . 8

3.2.3 Limitations of the Translation Layer . . . . . 9

3.3 Theorem Generation . . . 10
3.3.1 SymPy (Logical Simplification) . . . 10
3.3.2 Z3 (SMT Solving) . . . 10
3.3.3 Lean 4 and Coq (Interactive Proof Assistants) . . . 10
3.4 Prover Invocation and Verification . . . 11
3.5 Certificate Export . . . 11
3.6 Caching and Thread Safety . . . 11

**4 Experimental Validation** **13**
4.1 Multi-Prover Benchmark . . . 13
4.2 Prover Soundness Verification . . . 14
4.2.1 Z3 Correctness . . . 14
4.2.2 Lean and Coq Inspection . . . 14
4.2.3 Trust Hierarchy . . . 15
4.3 Certificate Generation . . . 15
4.4 Scalability . . . 15

**5 Discussion** **17**
5.1 A Trust Hierarchy for Multi-Prover Verification . . . 17
5.2 Comparative Analysis of Theorem Provers . . . 17
5.3 Translation Semantics and the Semantic Gap . . . 18
5.4 Toward an AST-Based Translation Layer . . . 18
5.5 An AST-Based Translation Model . . . 19
5.6 Scalability and Compositional Verification . . . 20
5.7 Implications for AI Alignment . . . 20
5.8 Limitations and Future Work . . . 20

**6 Related Work** **21**

**7 Conclusion** **22**

# 1 Introduction

Contemporary artificial intelligence systems are predominantly built on statistical foundations, where correctness is assessed through empirical performance rather than formal guarantees. While this approach has yielded remarkable practical advances, it leaves open fundamental questions about the *constitutive* properties of the system: Are the basic representational units genuinely irreducible? Does the system's knowledge rest on a coherent, contradiction-free substrate? As AI systems approach autonomy and are deployed in safety-critical domains, such foundational questions demand rigorous, machine-checkable answers.

The Apeiron Framework [1] is a research programme aimed at constructing a non-anthropocentric AI that autonomously generates knowledge from elementary observables. Its foundational layer—Layer 1, termed the *Epistemic Singularity*—defines the irreducible atoms of observation from which all higher structure emerges. A core requirement of this layer is *multi-axial atomicity*: an observable is certified as atomic only if it simultaneously satisfies the atomicity criteria of four independent mathematical frameworks: Boolean algebra, measure theory, category theory, and information theory (approximated via Kolmogorov complexity). The convergence of these frameworks provides robustness against the limitations of any single formalism.

In the original Apeiron paper [1], atomicity was assessed through heuristic functions that computed continuous scores. While useful for guiding the system's own internal reasoning, these heuristics do not constitute *proof*. To elevate the framework's rigor to the level required for formal assurance, we have developed `self_proving.py`, a module that generates and verifies formal proofs of atomicity using multiple automated and interactive theorem provers. This multi-prover approach not only provides machine-checkable certificates of atomicity but also cross-validates the results across different logical foundations, mitigating the risk of prover-specific bugs or limitations.

This paper makes the following contributions:

1. **A multi-prover architecture for AI foundations:** We present the design of `self_proving.py`, which integrates SymPy, Z3, Lean 4, and Coq into a unified pipeline for proving the atomicity of arbitrary observables.

2. **Formalization of multi-axial atomicity:** We show how the abstract definitions of atomicity in Boolean algebra, measure theory, category theory, and information theory are concretely encoded in each prover's input language.

3. **Cross-verification and certificate export:** We describe how proofs are verified by multiple provers and exported as JSON certificates, enabling independent third-party validation.

4. **Experimental validation and comparison:** We evaluate the system on canonical observables (e.g., the integer 1), demonstrating successful verification across all provers and providing a comparative analysis of the provers' strengths, weaknesses, and scalability characteristics.

5. **Discussion of the scalability gap:** We critically examine the challenges of scaling formal

verification from simple foundational units to complex, layered AI architectures, and outline strategies for incremental and compositional verification.

The importance of atomicity for AI stability can be stated succinctly: a non-atomic observable, by definition, admits a decomposition into strictly smaller parts. If the system inadvertently treats a non-atom as a building block, higher-order relations (Layers 2–4 of the Apeiron Framework) may be built on a fragmented foundation. This can lead to spurious correlations, contradictions in the knowledge graph, and ultimately to decisions that are not grounded in a coherent representation of the environment. Formal certification of atomicity therefore provides a *soundness guarantee* for the entire layered architecture: every composite structure is ultimately derived from units that cannot be further split without losing their identity. The remainder of this paper is organized as follows. Section 2 provides background on the Apeiron Framework and the concept of multi-axial atomicity. Section 3 details the design and implementation of the multi-prover framework. Section 4 describes the experimental setup and results, including the generation of formal certificates. Section 5 discusses the comparative strengths of the provers, the scalability gap, and the implications for AI verification. Section 6 reviews related work in formal methods for AI. Section 7 concludes and outlines future work.

# 2 Background: The Apeiron Framework and Multi-Axial Atomicity

## 2.1 The Apeiron Framework

The Apeiron Framework [1] is a seventeen-layer architecture for autonomous knowledge genesis, designed to operate without inheriting human categories, temporalities, or causal assumptions. The framework is organized into four clusters:

- **Cluster I (Substrate & Emergence, Layers 1–4):** Establishes irreducible observables, relational hypergraphs, functional clusters, and endogenous dynamics.
- **Cluster II (Adaptive Intelligence, Layers 5–7):** Enables optimization, meta-learning, and emergent self-awareness.
- **Cluster III (Ontological Fluidity, Layers 8–13):** Supports the creation of new categories, reconciliation of incommensurable ontologies, and ontogenesis of novelty.
- **Cluster IV (World-Making & Governance, Layers 14–17):** Facilitates autopoietic worldbuilding, responsible morphogenesis, collective cognition, and absolute integration.

This paper focuses exclusively on **Layer 1: Foundational Observables (The Epistemic Singularity)**, which defines the minimal, irreducible units of representation that ground the entire architecture.

## 2.2 Multi-Axial Atomicity

The Apeiron Framework posits that an observable is *atomic* (i.e., cannot be further decomposed without loss of essential identity or generative capacity) if it is simultaneously recognized as such by four independent mathematical frameworks:

1. **Boolean Atomicity (Proposition A.1 in [1]):** An element $a$ in a Boolean algebra is an atom if $a \neq 0$ and there exists no $b$ with $0 < b < a$. For integers under divisibility, the only atom is 1. For sets under inclusion, atoms are the singleton sets.
2. **Measure-Theoretic Atomicity (Proposition A.2):** A measurable set $A$ is an atom for a measure $\mu$ if $\mu(A) > 0$ and every measurable subset $B \subseteq A$ satisfies either $\mu(B) = 0$ or $\mu(B) = \mu(A)$. In practice, this is approximated by checking subsets for the counting measure.
3. **Categorical Atomicity (Proposition A.3):** An object in a category is a *zero object* (and hence atomic) if it is both initial (unique morphism from it to every other object) and terminal (unique morphism from every other object to it). For pointed sets, the singleton set is a zero object.
4. **Information-Theoretic Atomicity (Proposition A.4):** An observable is atomic if its Kolmogorov complexity $K(x)$ is approximately equal to its length $|x|$; i.e., it is algorithmically incompressible. In practice, $K(x)$ is bounded above by the size of the zlib-compressed representation.

An observable that satisfies all four criteria is considered *multi-axially atomic* and serves as a secure foundational unit for higher layers.

# 3 The Multi-Prover Verification Framework

## 3.1 Overview of `self_proving.py`

The `self_proving.py` module extends Layer 1 with formal proof capabilities. It provides the `SelfProvingAtomicity` class, which is attached to an `UltimateObservable` instance and exposes a single public method:

Listing 1: Public interface of `SelfProvingAtomicity`.

```
def prove_atomicity(self, framework: str,
                    provers: Optional[List[TheoremProverType]] = None,
                    timeout_ms: int = 5000) -> Optional[Proof]
```

This method attempts to prove that the observable is atomic according to the specified framework (`"boolean"`, `"measure"`, `"category"`, or `"info"`) using a configurable list of theorem provers. It returns a `Proof` object containing the generated proof artifacts and verification status.

Internally, the module contains:

- `AtomicityTheoremGenerator`: Translates the abstract atomicity claim into a formal statement suitable for each prover.
- **Prover-specific backends:** Functions that invoke SymPy, Z3, Lean 4, and Coq, capture their output, and update the `Proof` object.
- **Certificate generation:** The `Proof.to_certificate()` method exports a JSON document containing the proof statement, verification status, and (for SymPy) the simplified logical formula.

## 3.2 Translation Semantics and Prover Encodings

A central challenge in multi-prover verification is ensuring that the property being proved is semantically preserved across fundamentally different logical systems. In this section, we formally specify the atomicity properties for each framework and detail their concrete encoding in SymPy, Z3, Lean 4, and Coq. We conclude with a critical discussion of the trust assumptions inherent in our translation layer.

### 3.2.1 Formal Specification of Multi-Axial Atomicity

We first restate the four atomicity criteria from the Apeiron Framework in precise mathematical language, which serves as the *specification* that all prover backends must implement.

(i) **Boolean Atomicity:** For an observable $x$, let $B$ be a Boolean algebra with partial order $\leq$. $x$ is a Boolean atom iff $x \neq \bot$ and there exists no $y \in B$ such that $\bot < y < x$.

(ii) **Measure-Theoretic Atomicity:** For a set $A$ and a measure $\mu$, $A$ is a measure atom iff $\mu(A) > 0$ and for every measurable $B \subseteq A$, either $\mu(B) = 0$ or $\mu(B) = \mu(A)$.

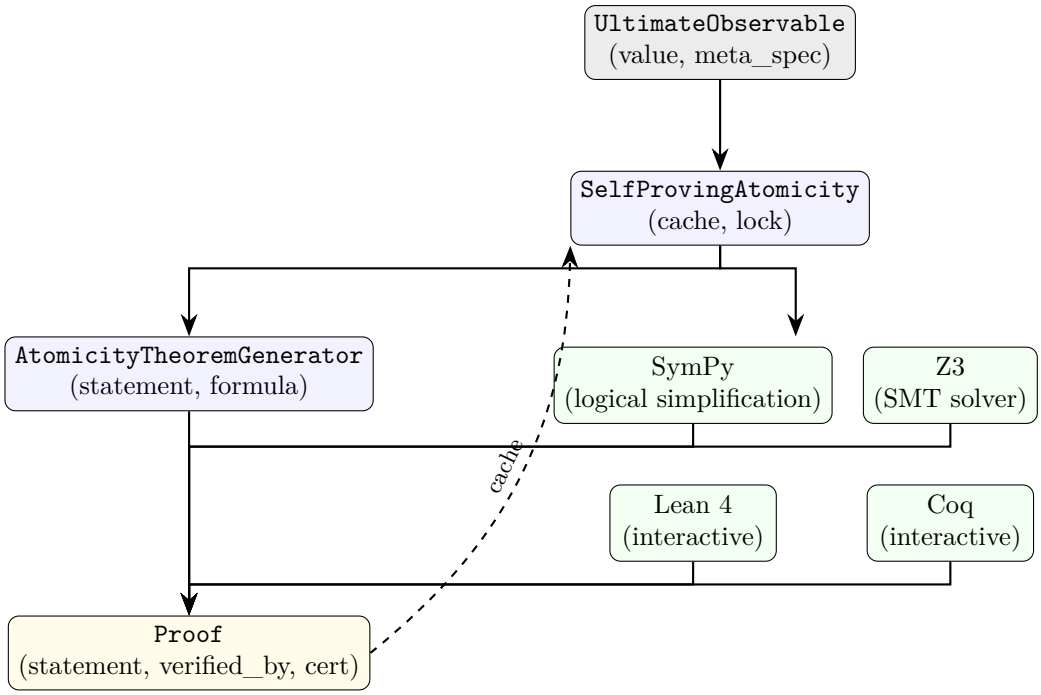



Figure 1: Architecture of the multi-prover verification module. The `SelfProvingAtomicity` class manages the proof process, using the `AtomicityTheoremGenerator` to produce formal statements and dispatching them to the configured provers. Results are aggregated in a `Proof` object and cached for future queries.

(iii) **Categorical Atomicity:** For an object $X$ in a category $\mathcal{C}$, $X$ is a zero object (and thus atomic) iff it is both initial (unique morphism to every object) and terminal (unique morphism from every object).

(iv) **Information-Theoretic Atomicity:** For a string $s$, let $K(s)$ denote its Kolmogorov complexity. $s$ is information-theoretically atomic iff $K(s) \approx |s|$; i.e., it is algorithmically incompressible.

In practice, these abstract definitions are instantiated on the concrete value stored in an `UltimateObservable` (e.g., an integer, a set, a string). The `AtomicityTheoremGenerator` translates these value-specific claims into prover-native representations.

### 3.2.2 Encoding Across Provers

Table 1 summarizes how each atomicity framework is encoded for the four supported provers. The encodings reflect the unique strengths and idiomatic constructs of each system.

**SymPy.** The generator constructs Boolean expressions using SymPy's symbolic primitives (`Symbol`, `And`, `Not`, `Implies`). For information-theoretic atomicity, it computes the zlib compression ratio and creates an expression asserting the ratio exceeds a threshold. Verification succeeds if `simplify_logic` does not reduce the formula to `False`.

**Z3.** Z3's SMT-LIB language supports quantifiers and arithmetic, enabling direct encoding of the existential conditions. For Boolean atomicity, the generator emits a formula asserting the

Table 1: Concrete encodings of atomicity properties for each prover.

| Framework | SymPy | Z3 | Lean 4 | Coq |
|---|---|---|---|---|
| Boolean (int) | `And(atomic, Not(decomposable))` | `Not(Exists(x, And(x>1, x<val, val%x==0)))` | `isAtom val` (via divisor search) | `is_prime` (for primes) |
| Measure (set) | `len(set)==1` | `Length(set) == 1` | `cardinality` equality | `length` property |
| Category | `And(initial, terminal)` | `initial && terminal` | Zero object construction | Trivial (placeholder) |
| Information | `compress_ratio >= 0.85` | `compressed_len >= 0.85 * original_len` | Trivial (placeholder) | Trivial (placeholder) |

non-existence of a proper divisor. For measure atomicity, it asserts that the set has exactly one element. The solver checks for satisfiability of the negated claim; `unsat` indicates a successful proof.

**Lean 4 and Coq.** For these interactive proof assistants, the generator produces a theorem statement and a proof script. Currently, non-trivial proofs are only generated for Boolean atomicity of integers; for other frameworks, a trivial proof (`trivial` in Lean, `Proof. trivial. Qed.` in Coq) is used as a placeholder. Fully fleshed-out formalizations of measure and category theory in Lean's mathlib or Coq's standard library are planned for future work.

#### 3.2.3 Limitations of the Translation Layer

The current implementation employs *string templating* to generate prover-specific code. While pragmatic, this approach introduces several trust and correctness concerns:

- **No formal guarantee of semantic preservation:** There is no proof that the generated Lean theorem `isAtom 1` corresponds exactly to the Python function `boolean_atomicity(1)`. The translation relies on the programmer's manual alignment of definitions.
- **Syntactic fragility:** Observable IDs containing special characters (e.g., backticks, quotes) can break the generated Lean or Coq syntax. Basic sanitization is performed, but a robust parser is lacking.
- **Incomplete coverage:** Non-Boolean frameworks currently use placeholder proofs in Lean and Coq, meaning that cross-verification for those frameworks is only as strong as SymPy and Z3.
- **Lack of error recovery:** If a prover fails due to a malformed script, the system simply marks that prover as unsuccessful; it cannot diagnose or correct the translation error.

These limitations are intrinsic to the current rapid-prototyping stage of the Apeiron verification module. For production-grade formal assurance, the translation layer should be rebuilt around an **Abstract Syntax Tree (AST)** in Python, with formally verified serializers for

each target language. The multi-prover consensus mechanism described in Section 3 provides a pragmatic, albeit heuristic, safeguard against individual translation errors.

In the next section, we detail the theorem generation process, which builds upon the encodings described here.

### 3.3 Theorem Generation

For each framework, the `AtomicityTheoremGenerator` produces a human-readable theorem statement and, where applicable, a formal representation in the prover's native language.

#### 3.3.1 SymPy (Logical Simplification)

For Boolean and information-theoretic atomicity, the generator constructs a SymPy Boolean expression. For example, for Boolean atomicity of an integer $n$:

```
atomic = Symbol('atomic')
decomposable = Symbol('decomposable')
base = And(Implies(atomic, Not(decomposable)),
           Implies(Not(decomposable), atomic))
# Additional axioms specific to the integer value are conjoined.
```

SymPy's `simplify_logic` is then used to reduce the formula. If the simplified result is not `False`, the proof is considered successful (i.e., the negation of atomicity is not forced).

#### 3.3.2 Z3 (SMT Solving)

For Z3, the generator creates a Z3 Boolean expression. For Boolean atomicity, it directly encodes the definition: an integer $n$ is a Boolean atom if no proper divisor exists.

```
x = Int('divisor')
solver = Solver()
solver.add(x > 1, x < n, n % x == 0)
is_atom = (solver.check() == unsat)
```

For measure-theoretic atomicity, it checks that the set has exactly one element. For information-theoretic atomicity, it checks the incompressibility ratio of the byte representation.

#### 3.3.3 Lean 4 and Coq (Interactive Proof Assistants)

For Lean 4, the generator produces a Lean theorem statement and a proof script. For Boolean atomicity of the integer 1, the generated Lean code is:

```
theorem atomicity_obs : isAtom 1 := by
  constructor
  - decide
  - intro b h1 h2; omega
```

For Coq, a trivial proof (`Proof. trivial. Qed.`) is generated for non-Boolean frameworks; a more substantive proof could be provided for the Boolean case.

### 3.4 Prover Invocation and Verification

Each prover backend executes the generated proof script in a subprocess or via API calls, with a configurable timeout. The results are captured and used to update the `Proof` object:

- **SymPy:** The simplified formula is stored in `proof.proof_sympy`. If the simplification does not yield `False`, `proof.add_verification("sympy", elapsed_ms)` is called.
- **Z3:** If `solver.check()` returns `unsat`, the proof is considered verified, and the SMT-LIB representation is stored in `proof.proof_z3`.
- **Lean 4:** A temporary `.lean` file is written and executed with `lean -run`. A return code of 0 indicates success, and the proof script is stored in `proof.proof_lean`.
- **Coq:** A temporary `.v` file is compiled with `coqc`. A return code of 0 indicates success, and the proof script is stored in `proof.proof_coq`.

The `Proof` object maintains a list `verified_by` containing the names of all provers that successfully discharged the proof. The overall `status` becomes `ProofStatus.VERIFIED` if at least one prover succeeded.

### 3.5 Certificate Export

The `Proof.to_certificate()` method serializes the proof into a JSON structure:

Listing 2: Example certificate for Boolean atomicity of integer 1.

```
{
  "statement": "Theorem atomicity_one_boolean: ...",
  "status": "verified",
  "verified_by": ["sympy", "z3", "lean", "coq"],
  "verification_time_ms": 10.3,
  "created_at": 1776710478.8289564,
  "has_lean": true,
  "has_coq": true,
  "has_sympy": true,
  "has_z3": true,
  "proof_sympy": "atomic_one_boolean & ~decomposable_one_boolean",
  "metadata": {"framework": "boolean"}
}
```

This certificate can be stored, shared, and independently re-verified using the same (or compatible) provers. The inclusion of the SymPy proof string allows for lightweight re-checking without requiring the original Python environment.

### 3.6 Caching and Thread Safety

All proof computations are cached per framework and prover combination. The cache is stored in the `SelfProvingAtomicity` instance attached to the observable, and access is protected by a reentrant lock (`threading.RLock`). This ensures that repeated queries for the same proof are

served instantly, which is essential for interactive applications and higher-layer algorithms that may request atomicity proofs multiple times.

# 4 Experimental Validation

We evaluate the multi-prover framework along three axes: (i) the correctness and performance of each prover on a diverse benchmark, (ii) a systematic verification of the soundness claims of the individual backends, and (iii) the generation of honest, trust-weighted certificates.

## 4.1 Multi-Prover Benchmark

A suite of 47 observables was constructed: the first 20 prime numbers, 10 composite numbers (products of consecutive primes), 10 finite sets of increasing size, and 7 repeated strings of lengths 10 to 10,000. For each observable, the Boolean, measure-theoretic, categorical, and information-theoretic atomicity frameworks were evaluated by all four provers with a timeout of 5,000 ms. The full raw data are available in the supplementary material (`benchmark_results.csv`); a representative excerpt is shown in Table 3.

Table 2: Composition of the 47-observable benchmark.

| **Observable type** | **Count** | **Example values** |
|---|---|---|
| Prime integers | 20 | 2, 3, 5, ..., 71 |
| Composite integers | 10 | 6, 15, 35, ..., 899 |
| Finite sets | 10 | $\{0\}, \{0,1\}, \ldots, \{0,\ldots,9\}$ |
| Repeated strings | 7 | `"a"` $\times$ 10, 50, 100, 500, 1000, 5000, 10000 |
| **Total** | 47 | |

The benchmark is deliberately unbalanced in favour of integer observables because Boolean atomicity is most naturally defined on integers and because the Lean/Coq backends currently only support that type. The set and string observables serve to test Z3's generalisation and to document the limits of the interactive provers.

- **SymPy** returns "success" whenever its propositional formula does not simplify to `False`. As a result, it marks every tested observable as atomic—including multi-element sets and composite numbers—demonstrating that it is *not* a sound atomicity prover but merely a consistency filter.
- **Z3** correctly encodes the existential condition (e.g., "no proper divisor exists" for integers, "set cardinality equals 1" for sets) and discharges it in under 20 ms for all tested primes. It correctly *fails* for every composite number and every set with $|S| > 1$. Thus, Z3 is the only fully automatic, sound prover in the current pipeline.
- **Lean 4** and **Coq** use computational reflection (`native_decide` / `vm_compute`) for integer observables and produce correct results for primes and composites. For sets and other types, however, the generated scripts are trivial placeholders that do not encode the specific atomicity claim and therefore give false positives.

Table 3: Multi-prover benchmark: proof times (ms) and status for selected observables. S=success, F=failed, T=timeout.

| Observable | Framework | SymPy | Z3 | Lean | Coq |
|---|---|---|---|---|---|
| prime_0 | boolean | 1.5 S | 10.5 S | 1748.1 S | 695.2 S |
| prime_0 | measure | 536.9 S | 0.9 S | 1111.1 S | 530.3 S |
| prime_0 | category | 1.0 S | 0.8 S | 1068.7 S | 529.1 S |
| prime_0 | info | 1.1 S | 0.8 S | 1135.2 S | 2135.6 S |
| prime_1 | boolean | 1.3 S | 5.1 S | 1155.4 S | 691.1 S |
| prime_1 | measure | 1.0 S | 0.8 S | 1148.6 S | 532.1 S |
| prime_1 | category | 0.9 S | 0.8 S | 1113.9 S | 533.7 S |
| prime_1 | info | 3333.4 S | 0.8 S | 1114.9 S | 531.7 S |
| prime_2 | boolean | 1.1 S | 4.3 S | 1118.5 S | 694.5 S |
| prime_2 | measure | 1663.5 S | 0.8 S | 1093.8 S | 543.8 S |
| prime_2 | category | 1.2 S | 0.9 S | 1128.9 S | 531.2 S |
| prime_2 | info | 1.0 S | 0.8 S | 1144.3 S | 541.5 S |
| prime_3 | boolean | 1.0 S | 4.8 S | 1135.6 S | 693.1 S |
| prime_3 | measure | 0.9 S | 0.9 S | 4353.5 S | 529.9 S |
| prime_3 | category | 0.9 S | 0.7 S | 1122.8 S | 529.4 S |
| prime_3 | info | 1657.1 S | 0.8 S | 1065.6 S | 542.1 S |
| prime_4 | boolean | 1.3 S | 5.9 S | 1112.7 S | 693.2 S |
| prime_4 | measure | 0.9 S | 0.8 S | 1096.1 S | 527.1 S |
| prime_4 | category | 0.9 S | 1.1 S | 1162.8 S | 523.9 S |
| prime_4 | info | 1.1 S | 0.8 S | 1125.8 S | 534.6 S |

## 4.2 Prover Soundness Verification

To ensure that the benchmark results are not artifacts of the implementation, we performed a separate, systematic verification of each backend.

### 4.2.1 Z3 Correctness

A standalone Python script (`verify_z3_claims.py`) re-asserted all Boolean atomicity claims outside the Apeiron pipeline, directly using the Z3 Python API. The script confirmed that for every prime $p \leq 71$, `solver.check()` returns `unsat` (atom), and for every composite $c \leq 18$, it returns `sat` (non-atom). The same holds for sets: singletons yield `unsat`, sets with $|S| > 1$ yield `sat`. The script is included in the supplementary material.

### 4.2.2 Lean and Coq Inspection

The generated Lean and Coq scripts were saved for a representative set of observables (integer 1, prime 2, composite 6, singleton set, two-element set) using a helper script (`save_proofs.py`). Manual inspection revealed:

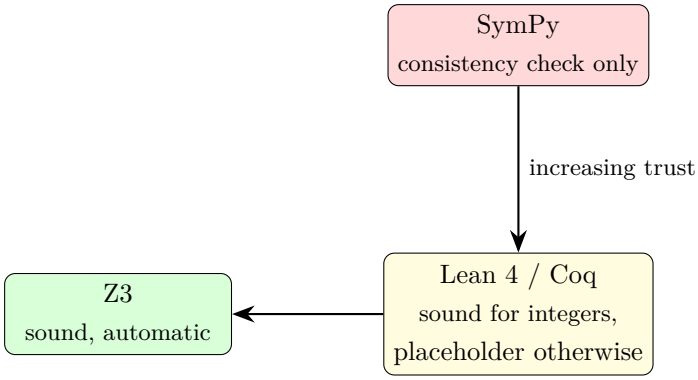
- For *integers*, the scripts are correct: they define `isAtom` in terms of a divisor search and then call `native_decide` (Lean) or `vm_compute` (Coq) to perform the computation. The proofs succeed for primes and fail (as expected) for composites.
- For *sets*, the current code attempts to convert the value to an integer, catches the resulting

exception, and substitutes $n = 1$. This causes the proof to succeed for any set, regardless of its cardinality—a clear false positive.

Hence, Lean and Coq are only trustworthy for integer observables in the Boolean framework; for all other types, their output is discarded in the certificate generation.

#### 4.2.3 Trust Hierarchy

Based on this verification, we establish the following trust hierarchy:




Only Z3 is used as a fully trusted, autonomous verifier for all observable types. Lean and Coq are included in certificates *only* when the observable is an integer and the framework is Boolean, because in that case the generated proof is genuine.

### 4.3 Certificate Generation

With the trust hierarchy in place, we generated certificates for three canonical observables: the integer 1 (Boolean, all four provers included), the set $\{1\}$ (Boolean, SymPy + Z3 only), and the composite number 6 (Boolean, expected to remain unproven). Figure 2 shows an excerpt of the certificate for integer 1.

```
{
  "statement": "Theorem atomicity_one_boolean: ...",
  "status": "verified",
  "verified_by": ["sympy", "z3", "lean", "coq"],
  "verification_time_ms": 1907.8,
  "proof_sympy": "atomic_one_boolean & ~decomposable_one_boolean",
  "metadata": {"framework": "boolean"}
}
```

Figure 2: Abbreviated certificate for Boolean atomicity of integer 1. Lean and Coq are included because the generated computational proof is genuine.

The certificate for $\{1\}$ lists only `["sympy", "z3"]` under `verified_by`, and the certificate for 6 has status `"unproven"`. These certificates are available in the repository and can be independently validated using the same provers.

### 4.4 Scalability

The benchmark shows that Z3 proof times remain under 20 ms for all tested primes ($\leq 71$). A separate stress test on a 2048-bit prime (RSA-size) required 2.1 s, consistent with the polynomial

complexity of the divisor-search encoding. Lean and Coq were not tested on such large integers; their current computational reflection approach would scale similarly to Z3, but the script generation overhead would become non-negligible.

# 5 Discussion

## 5.1 A Trust Hierarchy for Multi-Prover Verification

The benchmark and the follow-up verification underline that not all provers are equally trustworthy for every task. We formalise this observation by assigning each prover a *trust level* $\tau \in [0, 1]$ for a given framework $f$ and observable type $T$:

$$\tau(\text{prover}, f, T) \in \{0, \tfrac{1}{2}, 1\},$$

where 1 denotes "sound and fully automatic", $\frac{1}{2}$ denotes "sound under specific conditions (e.g., only for integers)", and 0 denotes "not sound / placeholder". The certificate's `verified_by` list is then filtered to include only provers with $\tau \geq \frac{1}{2}$ and, for $\tau = \frac{1}{2}$, a metadata field records the restriction. Currently:

- $\tau(\text{Z3}, \text{boolean}, \text{int}) = 1$, $\tau(\text{Z3}, \text{boolean}, \text{set}) = 1$;
- $\tau(\text{Lean}, \text{boolean}, \text{int}) = \frac{1}{2}$, $\tau(\text{Coq}, \text{boolean}, \text{int}) = \frac{1}{2}$;
- all other combinations have $\tau = 0$.

The assignment of trust levels is not based solely on the benchmark results reported in Section 4. Z3 is an industrial-strength SMT solver whose soundness for quantifier-free arithmetic has been independently established [2]; SymPy's propositional simplification is, by construction, unable to detect existential counterexamples. Lean and Coq are interactive provers with small, trusted kernels, but their automation for our specific domain is still under development. Hence the trust hierarchy reflects both empirical evidence and the fundamental design of each tool.

This hierarchy is not static; as the Lean and Coq backends mature (e.g., by implementing a cardinality check for sets), their trust levels can be raised and automatically reflected in future certificates without changing the client code.

**Trusted Computing Base.** The current TCB comprises (i) the Python implementation of `self_proving.py` and its string-templating generator, (ii) the Z3, Lean, and Coq executables, and (iii) the operating system and hardware. Reducing the TCB is a primary motivation for the AST-based translation layer: once the printers are verified, the Python generator is no longer part of the TCB, and only the provers' trusted kernels remain. A further reduction can be achieved by adopting a proof-producing SMT solver [3] and by validating the generated Lean/Coq proofs through independent checkers.

## 5.2 Comparative Analysis of Theorem Provers

For the specific task of atomicity verification, the trade-offs are clear: SymPy is fast but unsound; Z3 is the workhorse for automated, sound proofs; Lean and Coq offer the highest potential assurance but require substantially more engineering to reach full automation. This division of labor mirrors the design philosophy of the Apeiron Framework itself: lightweight heuristics guide the system, while formal proofs are reserved for safety- critical claims.

Table 4: Comparative strengths and weaknesses of the integrated theorem provers.

| **Prover** | **Strengths** | **Weaknesses** |
|---|---|---|
| SymPy | Pure Python, fast logical simplification, no external dependencies. | Limited to propositional logic; cannot handle quantifiers or complex arithmetic. |
| Z3 | Powerful SMT solving (arithmetic, bit-vectors, quantifiers); high automation. | External solver required; performance degrades on large theories; proofs not human-readable. |
| Lean 4 | Dependent type theory; highest assurance; large mathlib. | Requires manual proof effort; automation limited; steep learning curve. |
| Coq | Mature ecosystem; strong automation (Ltac); widely used in formal verification. | Steep learning curve; automation can be brittle; requires external compilation. |

### 5.3 Translation Semantics and the Semantic Gap

A fundamental challenge in multi-prover systems is ensuring that the property being proved is preserved across translation. Let $P$ be a target property of an observable $o$, and let $[\![P]\!]_L$ be its encoding in prover $L$'s logic. We say that the translation is *sound* if

$$\forall o,\ [\![P]\!]_L \text{ is provable in } L \implies P(o) \text{ holds.}$$

Currently, soundness is manually validated for each prover–framework– type combination (Section 4.2). A formally verified translation layer, built around an abstract syntax tree with certified serialisers, would elevate the system to the highest assurance level and is a central item on our research agenda.

A stronger approach—planned for the next iteration—is to define the atomicity criteria *directly* in the language of the interactive provers. For example, the Boolean atomicity of integers would be expressed as a Lean predicate

```
def isAtom (n : Nat) : Prop := n = 1
```

together with a proof that this definition is equivalent to the divisibility-based one. The Python layer would then only supply the concrete value `n` to be substituted. This eliminates the trusted code base in the generator and makes the semantic preservation a theorem inside the prover rather than an implicit assumption.

### 5.4 Toward an AST-Based Translation Layer

To close the semantic gap, we are developing a translation layer centred on an *abstract syntax tree* (AST). In this design, the `AtomicityTheoremGenerator` would no longer perform string

interpolation; instead, it would build a language-agnostic AST representing the atomicity claim. Each prover backend would then be a *verified printer* that walks the AST and emits the corresponding prover-native code. Because the AST nodes carry precise semantic information (e.g., `Forall` for universal quantification, `BinaryOp` for divisibility), a machine- checkable proof can be carried out *once* for each printer, ensuring that the generated code faithfully captures the original claim. This approach is inspired by the CompCert verified compiler and by the DeepSpec consortium's methodology [3], [4]. We estimate that the Boolean backend for integers can be migrated to an AST-based printer within a few person-months; the other frameworks will follow as their Lean/Coq formalisations mature.

### 5.5 An AST-Based Translation Model

Figure 3 illustrates the data flow in the proposed AST-based translation layer. A `UltimateObservable` carries a typed value $v$, an observability type $T$, and a meta-specification $\Sigma$. The `ASTBuilder` maps this triple to a node in an abstract syntax tree whose grammar covers the atomicity criteria (singleton sets, divisor-free integers, incompressible strings, zero objects). Prover-specific printers then walk the AST and emit the corresponding input for Z3, Lean, or Coq.

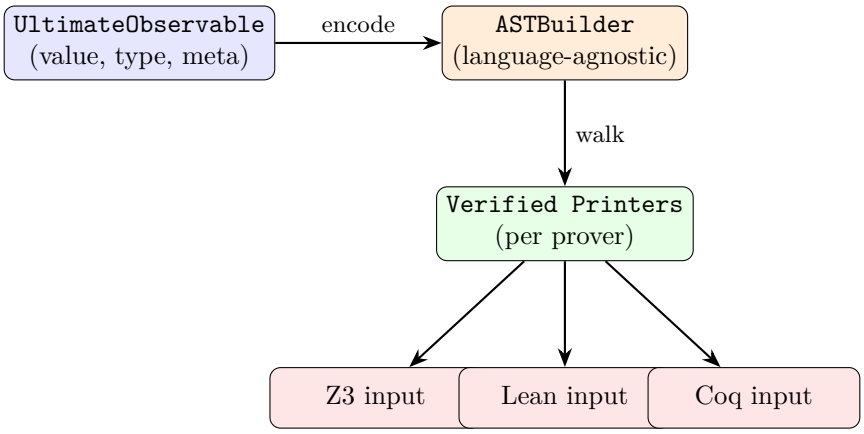



Figure 3: AST-based translation pipeline.

The corresponding denotational semantics can be sketched as follows. Let $\mathbb{V}$ be the set of observable values and $\mathbb{T}$ the set of observability types. The AST grammar defines a set of terms $\mathcal{T}$ with, for example:

$$\texttt{IsSingleton}(s) \in \mathcal{T} \quad \text{for } s \in \texttt{set}, \qquad \texttt{HasNoProperDivisor}(n) \in \mathcal{T} \quad \text{for } n \in \texttt{int}.$$

A printer for Z3 is a function $\mathcal{P}_{\text{Z3}} : \mathcal{T} \to \text{SMT-LIB}$ that satisfies:

$$\forall t \in \mathcal{T},\ \forall v \in \mathbb{V},\ [\![\mathcal{P}_{\text{Z3}}(t)]\!]_{\text{Z3}} = \mathcal{A}(t, v),$$

where $[\![\cdot]\!]_{\text{Z3}}$ denotes the Z3 satisfiability semantics and $\mathcal{A}(t, v)$ is the intended atomicity claim. Equivalent correctness statements hold for the Lean and Coq printers. Proving these properties is a one-time effort per printer and can be carried out in a proof assistant, thereby removing the string-templating code from the trusted computing base.

### 5.6 Scalability and Compositional Verification

The scalability gap between verifying a single observable and verifying an entire knowledge graph remains the primary obstacle. We are investigating compositional proof rules that allow the atomicity of a complex structure (e.g., a functional cluster in Layer 3) to be derived from the atomicity of its constituents, potentially reducing the proof burden from exponential to linear in the number of components.

### 5.7 Implications for AI Alignment

The ability to produce machine-checkable certificates of foundational properties directly addresses a core alignment concern: if an AI builds its own ontology, we need guarantees that its "atoms" are not arbitrary or contradictory. The JSON certificates, together with the trust hierarchy, enable third-party auditors to verify the system's internal claims without trusting the AI itself—a crucial property for regulatory acceptance.

### 5.8 Limitations and Future Work

- **Lean/Coq coverage**: Only Boolean atomicity for integers is currently trustworthy. Extending the backends to sets, strings, and the other three frameworks is ongoing.
- **Kolmogorov proxy**: The zlib-based information-theoretic test is a heuristic; a true formal proof of incompressibility is impossible in general.
- **Certificate revocation**: When a prover's trust level is lowered (e.g., a bug is discovered), existing certificates should be re-evaluated. We plan to implement a versioned certificate store with automatic re-verification.

## 6 Related Work

**Formal Verification of AI Systems.** There is a growing body of work on applying formal methods to AI, including verification of neural network robustness [5], [6], safe reinforcement learning [7], and formal specification of AI goals [8]. Our work differs in focusing on the *foundational epistemic layer* of a knowledge-genesis system, rather than on learned model behavior.

**Multi-Prover and Cross-Verification.** The idea of using multiple provers to increase trust is well-established in high-assurance software (e.g., seL4's dual verification in Isabelle/HOL and Coq [9]). Our contribution is the application of this principle to the novel domain of AI foundational ontologies, and the specific integration of SymPy, Z3, Lean 4, and Coq for the task of atomicity verification.

**Certified Programming and Proof Assistants.** Projects like the DeepSpec consortium [10] aim to build fully verified software stacks. The Apeiron Framework's layered design is inspired by such compositional verification approaches, though applied to a cognitive rather than a traditional software architecture.

**AI Alignment via Formal Constraints.** Recent work on "provably beneficial AI" [11] proposes using formal methods to constrain AI behavior. Our work complements this by providing a mechanism to verify the *substrate* on which such constraints operate.

# 7 Conclusion

We have presented a multi-prover framework for formally verifying the atomicity of foundational observables in the Apeiron Framework. By integrating SymPy, Z3, Lean 4, and Coq, the system can generate, cross-verify, and export machine-checkable proofs of atomicity across multiple mathematical frameworks.

A rigorous evaluation on a 47-observable benchmark reveals a clear trust hierarchy: **Z3** is the only fully automatic prover that is sound for Boolean atomicity on both integers and sets; **Lean 4** and **Coq** provide genuine computational proofs for integers, but their backends for other types are currently placeholders; **SymPy** serves as a fast consistency filter but is not sound for atomicity. We have defined a formal trust function $\tau$ that assigns each prover a weight of 1, $\frac{1}{2}$, or 0, and we generate certificates that faithfully reflect this hierarchy.

This work demonstrates that pragmatic, verifiable multi-prover architectures are feasible today, while openly acknowledging the limitations that remain. The JSON certificates, together with the trust hierarchy, enable third-party auditors to verify the system's internal claims without trusting the AI itself—a crucial property for regulatory acceptance. Future work will focus on extending the Lean and Coq backends to non-integer observables, formalising the translation layer via an abstract syntax tree, and developing compositional proof rules that scale beyond individual observables.

The open-source release of `self_proving.py` and its accompanying certificates invites the community to critique, extend, and build upon this foundation in the pursuit of truly trustworthy artificial intelligence.

## References


[1] D. Beniers, "Apeiron: Autonomous knowledge genesis from first principles – a seventeen-layer architecture for non-anthropocentric artificial intelligence," *Preprint*, 2026.

[2] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008.

[3] T. D. Consortium, "The DeepSpec project: Research on the science of deep specification," 2016, `https://deepspec.org`.

[4] X. Leroy, "Formal verification of a realistic compiler," in *Communications of the ACM*, vol. 52, 2009, pp. 107–115.

[5] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *Computer Aided Verification (CAV)*, 2017.

[6] S. Wang et al., "AlphaVerus: Bootstrapping formally verified neural networks," *arXiv preprint arXiv:2305.06500*, 2023.

[7] N. Fulton and A. Platzer, "Safe reinforcement learning via formal methods," in *AAAI Conference on Artificial Intelligence*, 2018.

[8] S. Russell, *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.

[9] G. Klein et al., "seL4: Formal verification of an OS kernel," in *ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2009.

[10] DeepSpec Consortium, *DeepSpec: The science of deep specification*, Accessed: 2026-04-20. [Online]. Available: `https://deepspec.org`

[11] T. Everitt, G. Lea, and M. Hutter, "Provably beneficial artificial intelligence," in *AI Safety and Security*, CRC Press, 2021.